COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	October 14, 2026	**DATE SUBMITTED:**	October 16, 2026
PAIR MEMBERS:	Thomas Joseph T. Astudillo Johrel Louie T. Silao	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. _12.1_ Algorithmic Strategies

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Strategy	Algorithm	Analysis
Recursion	Quick Sort	It is the process of breaking down a large array into smaller parts and this process will repeat until it no longer requires sorting.
Brute Force	Linear Search	Linear search works using brute force because it scans and checks every element in a data structure such as an array one by one until it eventually finds the target value or reaches the end of the structure without finding a match.
Back Tracking	Depth-First Search	This works by moving through choices, which continue to move forward in a single path as deeply as possible, making decision after decision.
Greedy	Selection Sort	Selection Sort works using a greedy strategy because it splits the array into sorted and unsorted portions. It then repeatedly scans through the unsorted section to find the smallest element, and then places it into its correct sorted position.
Divide-And-Conquer	Merge Sort	Merge sort works using divide-and-conquer because it recursively splits an array into smaller halves until each contains a single element, and then

		merges those sorted sub-arrays back together to form a fully sorted array.

Table 12-1. Algorithmic Strategies and Examples

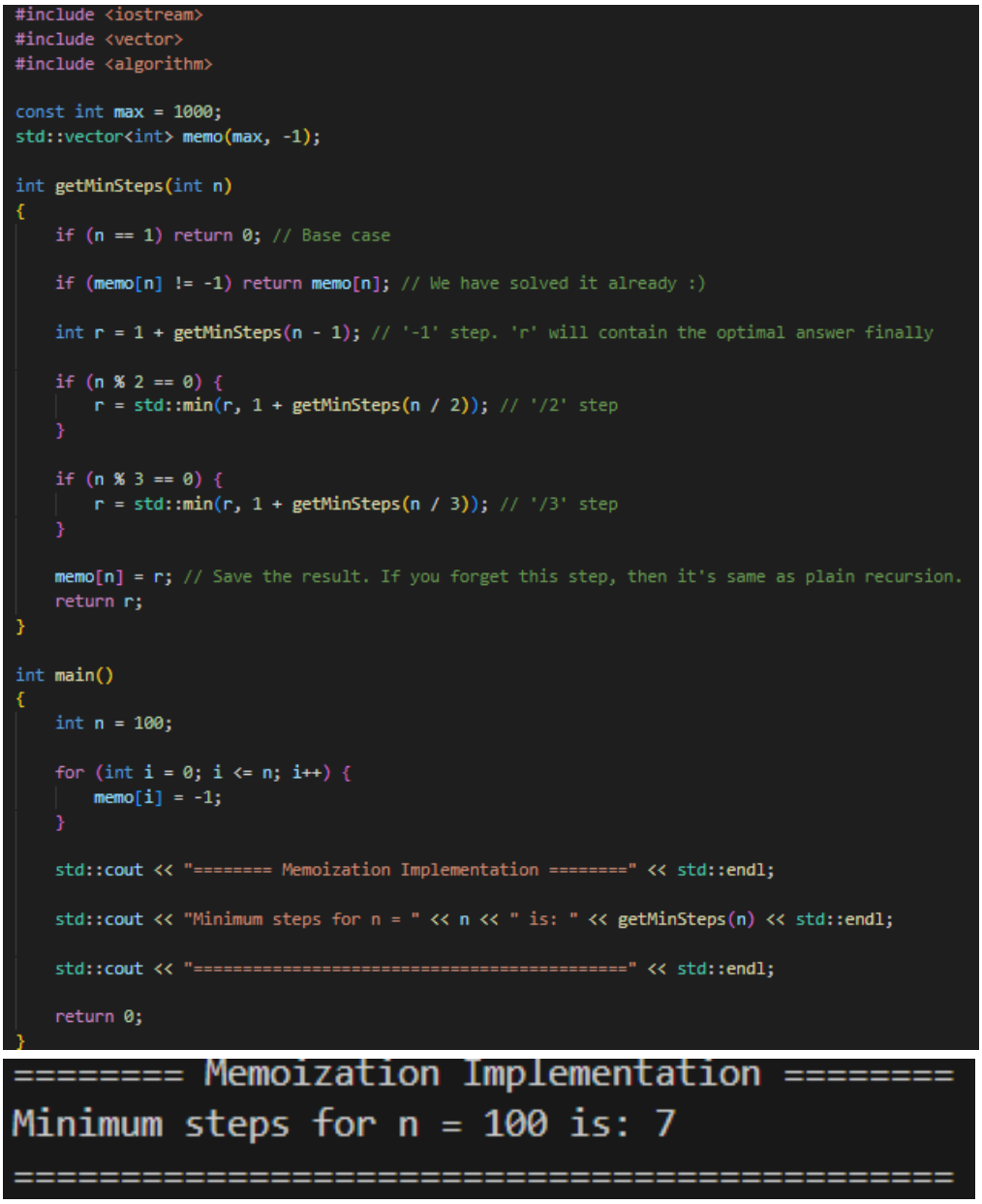
Screenshot + Output	
Analysis	This memoization implementation breaks down the ‘n = 100’ into smaller parts. Whenever an overlapping repetition occurs, the output is regained in O(1) time which reduces the time complexity from exponential to linear time complexity.

Table 12-2. Memoization Implementation

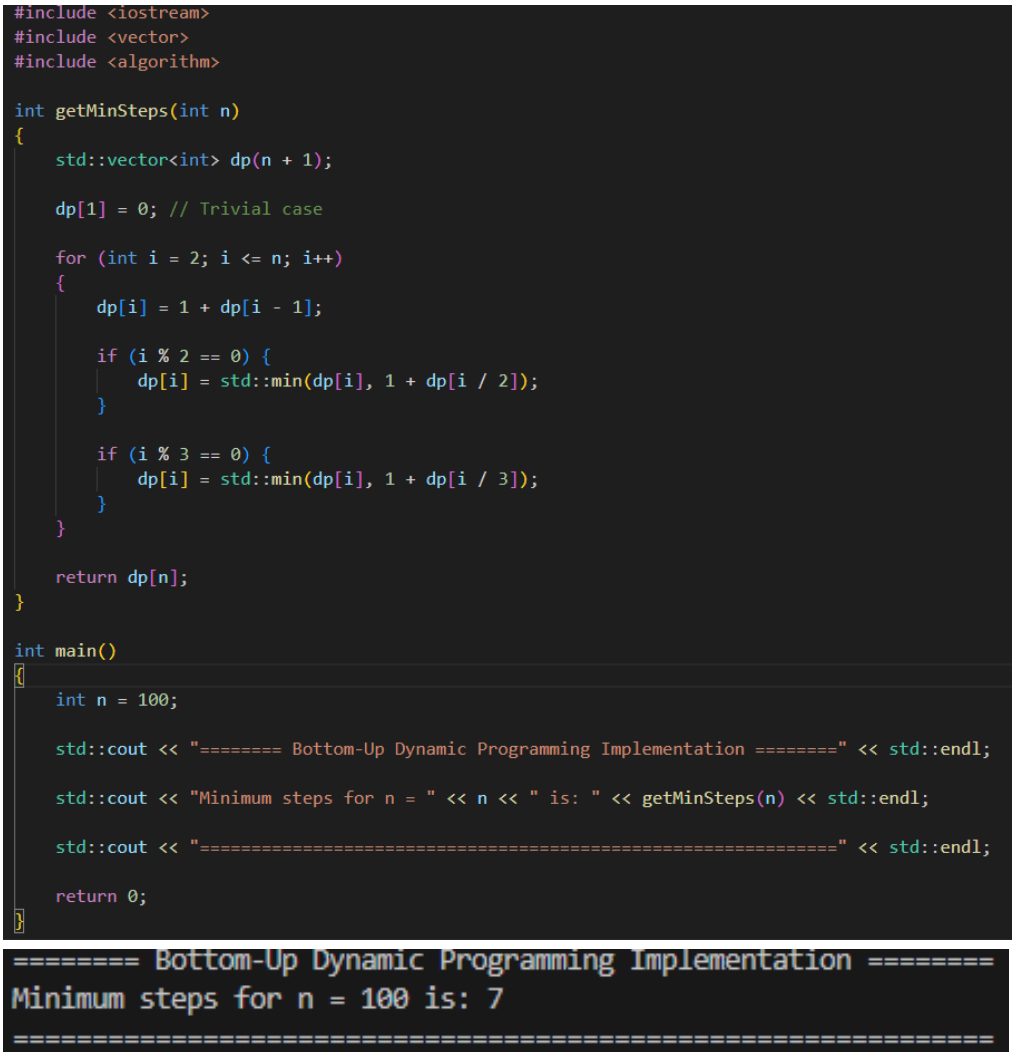
Screenshot + Output	
Analysis	The Bottom-Up Dynamic Program algorithm uses a tabulation strategy in order to solve sub-problems starting from the smallest case moving all the way up to the value of n, allowing each number to determine its minimum steps using a time complexity of O(1), meaning that it instantly reuses previously stored data in the table rather than having to re-calculate it repeatedly.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell

Description:
Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial

Pseudocode (+ Solving Problem by Hand)

Pseudocode:

```
START
FUNCTION countPaths(matrix, m, n, cost):
   // Base Case 1: Out of matrix boundaries or negative remaining cost
   IF m < 0 OR n < 0 OR cost < 0 THEN
      RETURN 0

   // Base Case 2: Reached starting cell (0,0)
   IF m == 0 AND n == 0 THEN
      IF cost == matrix[0][0] THEN
         RETURN 1
      ELSE
         RETURN 0

   // Recursive Step: Sum paths coming from Top (m-1, n) and Left (m, n-1)
   remainingCost = cost - matrix[m][n]
```

```
    RETURN countPaths(matrix, m - 1, n, remainingCost) +
           countPaths(matrix, m, n - 1, remainingCost)
END
```

Manual Counting

- **Path 1:**
  (0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2) -> (3,2) -> (3,3)
  $4 + 7 + 1 + 3 + 1 + 6 + 3 = 25$

- **Path 2:**
  $(0, 0) \rightarrow (1, 0) \rightarrow (1, 1) \rightarrow (1, 2) \rightarrow (2, 2) \rightarrow (2, 3) \rightarrow (3, 3)$
  $4 + 5 + 7 + 3 + 1 + 2 + 3 = 25$

Working C++ Code

```cpp
#include <iostream>
#include <vector>
#include <algorithm>


// FUNCTION countPaths(matrix, m, n, cost):
int countPaths(const std::vector<std::vector<int>>& matrix, int m, int n, int cost) {

    /* Base Case 1: Out of matrix boundaries or negative remaining cost
     IF m < 0 OR n < 0 OR cost < 0 THEN
       RETURN 0 */


    if (m < 0 || n < 0 || cost < 0) {
        return 0;
    }

    /* Base Case 2: Reached starting cell (0,0)
    IF m == 0 AND n == 0 THEN
        IF cost == matrix[0][0] THEN
            RETURN 1
        ELSE
            RETURN 0 */

    if (m == 0 && n == 0) {
        if (cost == matrix[0][0]) {
            return 1;
        } else {
            return 0;
        }
    }

    /* Recursive Step: Sum paths coming from Top (m-1, n) and Left (m, n-1)
    remainingCost = cost - matrix[m][n]
    */

    int remainingCost = cost - matrix[m][n];
    /* RETURN
        countPaths(matrix, m - 1, n, remainingCost) +
        countPaths(matrix, m, n - 1, remainingCost)
    END */
    return countPaths(matrix, m - 1, n, remainingCost) + countPaths(matrix, m, n - 1, remainingCost);


}
```

```cpp
int main() {
    // Initialize the 4x4 Vector
    // Insert the given Values
    std::vector<std::vector<int>> matrix = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    // Get rows
    int rows = matrix.size();
    // Get Colums
    int columns = matrix[0].size();
    // Initialize the target Cost
    int targetCost = 26;

    // Initialize the Path to get the target cost
    int Paths = countPaths(matrix, rows - 1, columns - 1, targetCost);

    // Output
    std::cout << "========= Matrix Path Cost Problem =========" << std::endl;
    std::cout << "Number of paths with cost " << targetCost << " is: " << Paths << std::endl;
    std::cout << "============================================" << std::endl;


    return 0;
}
```

Analysis of working code

Analysis of Astudillo:

- This code utilizes Recursive Algorithm along with Backtracking to explore all potential routes from the end of the cell all the way back to the starting position. By implementing Recursive function calls, the program breaks down the matrix into smaller subproblems. Not only that, but through the Backtracking function calls, it evaluates all of the boundaries along with its constraints in order to discard any invalid routes, such as those who exceed the target cost of 25 or values that go out of bounds, in which it backtracks to its previous position and explores alternative paths moving backward from the bottom-right destination to the top-left starting cell. These functions are essential in order to find the target cost, as they are able to check and thoroughly examine each potential route and combination, thus allowing the program to accurately find every valid path that matches the target cost.

Analysis of Silao:

- In this program, this solves the matrix problem path cost using the algorithmic strategies. The function will evaluate every possible route by exploring two movements at each step. In order to establish the base case properly to handle boundary conditions, it must return to zero if the traversal moves out of bounds or if the remaining cost drops zero. The recursive step subtracts the current cell's value from the target and collects the path counts from both directions.

Screenshots of demonstration



Conclusion

Conclusion of Astudillo:

- Based on the Hands-on Activity, that we performed today, I was able to understand and utilize the various basic algorithm strategies, such as recursion, brute force, backtracking, greedy, as well as divide-and-conquer in order to analyze, label, and create programs that solve time-complexity-based problems. More specifically, I was able to utilize the concepts of memoization and bottom-up dynamic programming approaches in order to determine the minimum steps it would take in order for the program to reach the target value of n. Through this process, I gained a deeper understanding of how to handle overlapping subproblems and reduce time complexity from exponential to linear or constant time. Not only that but the supplementary activity allowed us to understand the process and application of recursive logic, optimal substructures, and boundary checks to count the number of valid paths in a matrix with a specified target value.

  Overall, I believe we performed quite well in this hands-on activity. While we encountered some initial challenges with debugging and mapping out the recursive steps for the matrix path problem, my pair and I were able to collaborate, trace the paths manually, and successfully complete the required tasks. This activity helped me assess my understanding of basic algorithms and the skills I need to improve in order to fully understand the process of dynamic programming, recursive problem decomposition, and time complexity optimization.

Conclusion of Silao

- In this hands-on activity, I was able to learn and understand the algorithmic strategies. Additionally, I was able to apply and implement various algorithmic strategies and solve problems using the algorithmic strategies that I learned. Although we ran into some errors while finishing the activity, we were able to solve it and collaborate with each other in order to demonstrate the expected output of the activity. In the end, we were able to finish the activity and I intend to apply my learnings for future activities.

“I affirm that we will not give or receive any unauthorized help on this activity and that all work will be our own.”

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.